You can ensure that a candidate method is not treated as an action method by decorating it with the `[NonAction]` attribute.

Listing 19.2 The `HomeController` for the default MVC app

```
public class HomeController : Controller                                    ❶
{
    private readonly ILogger<HomeController> _logger;
    public HomeController(Ilogger<HomeController> logger)
    {
        _logger = logger;
    }

    public IActionResult Index()                                            ❷
    {
        return View();                                                      ❸
    }

    public IActionResult Privacy()
    {
        return View();
    }

    [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None,     ❹
         NoStore = true)]                                                   ❹
    public IActionResult Error()
    {
        return View(new ErrorViewModel                                      ❺
        {                                                                   ❺
            RequestId = Activity.Current?.Id                                ❺
                ?? HttpContext.TraceIdentifier                              ❺
        });
    }
}
```

❶ MVC Controllers often inherit from the Controller base class.

❷ Action methods are the endpoints that run in response to requests.

❸ Returning View() renders a Razor view.

❹ You can apply filters to actions, as you'll learn in chapters 21 and 22.

❺ Any object returned with View is passed to the Razor view as a view model.